SENAI

Ponte entre aula anterior e atual

Na aula anterior, a equipe **documentou decisões**; hoje, **conecta decisão** com **uso real**.

- Aula 07: **registrar** decisão, ADR, diagrama e rastreabilidade.
- Aula 08: **partir** de uma **necessidade do usuário** e **construir uma fatia mínima**.
- A entrega precisa mostrar ligação entre requisito, interação, interface, modelo, arquitetura e código.
- **Ideia central**: o usuário não vê a arquitetura, mas sente suas consequências.

Aula 08

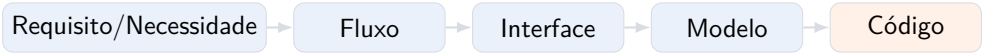
Fluxo do usuário, requisito, interface, modelo, arquitetura e código

Como sair de um requisito simples e chegar a uma funcionalidade coerente no projeto.

SENAI

Objetivo para o final da Aula 08

Executar uma parte mínima do projeto.



- Um requisito simples, mas que atravessa várias partes do software.
- Não será fazer muitas telas; é fazer uma parte coerente de ponta a ponta.
- Lembrando que para o Design de Software o resultado precisa ser demonstrável e rastreável.

Pergunta-guia: o que mudou para o usuário e onde isso aparece no projeto?

SENAI

Agenda

A aula trata de interface, notação, modelagem e ferramentas de representação.

1. Fluxo do usuário
2. Design de **interfaces de usuários**.
3. **Notações** de design de software.
4. **Modelagem**.
5. **Linguagens** e **ferramentas** aplicadas à representação da solução.
6. Relação entre interface, responsabilidades, modelos e arquitetura.

A atividade da aula aplica exatamente esses elementos no projeto do grupo.

SENAI

Por que começar pelo usuário?

Porque a **funcionalidade** existe para **resolver uma necessidade, ou seja, um requisito**.

- O usuário **enxerga** tarefas: cadastrar, solicitar, acompanhar, confirmar, cancelar.
- O sistema **responde** com telas, mensagens, regras e dados.
- A equipe **traduz essa experiência** para modelos e código.
- Quando a equipe ignora o usuário, a interface vira um formulário sem sentido.

Levantadas as necessidades o quê deve-se fazer?

Antes da solução, registre o problema ou necessidade.

Decompondo uma necessidade/requisito

Escolha **um requisito/necessidade** pequeno e claro.

Exemplo: o cliente precisa acompanhar o status de uma entrega para saber se ela está em preparação, em transporte ou entregue.

- Evite **requisitos gigantes**, como "gerenciar todo o sistema".
- Escreva com usuário/ator, verbo principal e resultado/informação desejada.

Leitura do requisito

Quem?	Cliente
Faz o quê?	Acompanhar
O quê?	Status da entrega
Resultado esperado	Visualizar a situação atual da entrega
Valores possíveis	Em preparação; em transporte; entregue

Não confunda os verbos

precisa	Indica necessidade; não é a funcionalidade.
acompanhar	É o verbo principal da funcionalidade.
saber	Explica por que o cliente quer acompanhar.
está	Apenas descreve o estado da entrega.

Um bom requisito cabe em poucas linhas e pode virar uma atividade prática.

SENAI

Problema vem antes da solução

Não comece escolhendo tecnologia.

- Ruim: “vamos criar uma tela em React”.
- Melhor: “o cliente não sabe em qual etapa está a entrega”.
- Depois disso, **a equipe escolhe** interface modelo e implementação.
- A tecnologia é meio; a necessidade do usuário é o ponto de partida.

Frase para orientar alunos: primeiro problema, depois solução.

SENAI

1. Fluxo do usuário

Mostra o caminho que a pessoa percorre para cumprir uma tarefa.

- Quem inicia a ação?
- Qual informação a pessoa fornece?
- O que o sistema valida?
- Qual resposta o sistema apresenta?
- Onde a tarefa termina com sucesso ou erro?

Fluxo é a história da interação, não a estrutura interna do código.

SENAI

Exemplo de fluxo simples

TAREFA: Acompanhar entrega.



Todo fluxo deve ter 1) início, 2) validação, 3) resposta e 4) fim.

- Caminho principal: código válido e status encontrado.
- Caminho alternativo: código inexistente ou pedido sem status.
- Resultado esperado: usuário entende a situação da entrega.

SENAI

2. Design de interface de usuário

É o ponto de contato entre pessoa e sistema.

- Uma interface pode ser tela, formulário, terminal, página web ou API consumida por outra aplicação.
- A interface precisa mostrar as informações certas no momento certo.
- Interface boa não é apenas bonita; é compreensível, útil e coerente com o processo.
- O protótipo pode ser simples: papel, draw.io, Figma, Canva, PowerPoint ou imagem no README.

Na aula, basta wireframe/protótipo simples e explicável.

SENAI

Wireframe

É o **rascunho** estrutural da tela.

- Mostra campos, botões, mensagens e organização básica.
- Não precisa de design visual sofisticado.
- Ajuda a equipe a discutir antes de codificar.
- Deve estar ligado ao requisito e ao fluxo escolhido.

Wireframe responde: o usuário verá e fará o quê?

SENAI

Exemplo de wireframe textual

Consulta de status da entrega.

```
+-----------------------------------------+
| NexusLog - Acompanhar entrega           |
| Codigo do pedido: [__________] [Consultar]|
|                                         |
| Status atual: EM_TRANSPORTE             |
| Ultima atualizacao: 14/09/2026 19:40    |
| Mensagem: Sua entrega esta a caminho.   |
+-----------------------------------------+
```

- Campo de entrada: código do pedido.
- Ação: consultar.
- Saída: status, data e mensagem compreensível.

Um wireframe simples já orienta modelo, componente e código.

SENAI

3. Notações de design de software

Notação é uma forma padronizada de representar uma ideia.

- Texto livre: fácil, mas pode ser ambíguo.
- Tabela: boa para rastreabilidade.
- UML: boa para classes, objetos, casos de uso e interações.
- Mermaid: útil para diagramas versionados no GitHub.
- C4: útil para arquitetura em níveis de detalhe.

Escolha a notação que explica melhor o problema da aula.

SENAI

UML de forma simples

UML é linguagem visual para comunicar partes do software.

- Diagrama de casos de uso: mostra atores e objetivos.
- Diagrama de classes: mostra conceitos, atributos, operações e relações.
- Diagrama de sequência: mostra mensagens ao longo do tempo.
- Diagrama de atividades: mostra etapas de um processo.

Para essa aula, escolha o diagrama que conecta melhor fluxo, modelo e código.

SENAI

4. Modelagem — o que é um modelo?

Modelo é uma representação simplificada de uma parte do software.

- Modelar ajuda a equipe a entender o sistema antes ou durante a implementação.
- O modelo não é o sistema completo; ele mostra apenas o que é importante naquele momento.
- Podemos representar o que existe, o que acontece e como as partes do sistema estão organizadas.
- UML e Mermaid são formas de representar esses modelos.

Modelar é representar uma parte do sistema para facilitar o entendimento antes de olhar diretamente para o código.

SENAI

4.1 Diagrama de classes

Mostra o que existe no sistema e como os elementos se relacionam.

- Classe representa um conceito importante do sistema, como Cliente, Pedido ou Entrega.
- Atributos representam informações armazenadas, como código, data ou status.
- Operações representam ações que o objeto pode realizar.
- Associações mostram como as classes se relacionam.
- Na Aula 08, modele apenas as classes relacionadas à funcionalidade escolhida.

Pergunta para lembrar: quais elementos precisam existir para essa funcionalidade funcionar?

SENAI

Exemplo — diagrama de classes

Consulta de status de uma entrega no NexusLog.

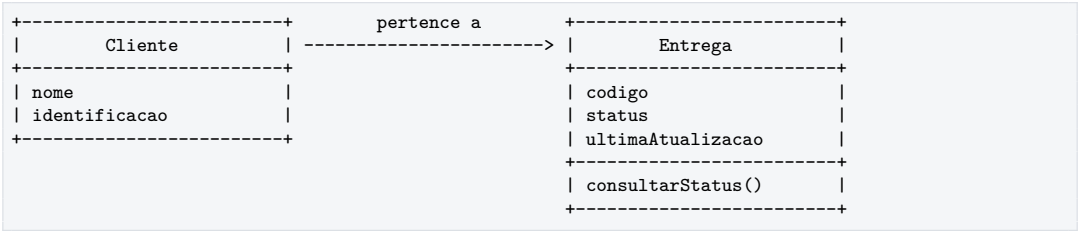


Diagrama de classes responde principalmente: o que existe no sistema?

SENAI

4.2 Diagrama de sequência

Mostra o que acontece quando uma funcionalidade é executada.

- Mostra quem participa da execução da funcionalidade.
- Mostra quem chama quem.
- Mostra a ordem em que as mensagens acontecem.
- Ajuda a ligar interface, serviço, repositório e resultado apresentado ao usuário.

Pergunta para lembrar: quem conversa com quem e em qual ordem?

SENAI

Exemplo — diagrama de sequência

Consulta de status da entrega.

```
Cliente -> TelaConsultaStatus: informa codigo
TelaConsultaStatus -> EntregaService: consultarStatus(codigo)
EntregaService -> EntregaRepository: buscarPorCodigo(codigo)
EntregaRepository --> EntregaService: entrega
EntregaService --> TelaConsultaStatus: status formatado
TelaConsultaStatus --> Cliente: exibe status
```

Diagrama de sequência responde principalmente: o que acontece durante a execução?

SENAI

4.3 Diagrama de componentes

Mostra como as partes maiores do sistema estão organizadas.

- Componente representa uma parte maior do sistema com responsabilidade definida.
- Um componente pode conter várias classes e serviços.
- Componentes se comunicam por interfaces, chamadas ou eventos.
- O diagrama ajuda a identificar onde uma funcionalidade deve ficar na arquitetura.
- Não mostra cada classe; mostra as principais partes da solução.

Pergunta para lembrar: em qual parte do sistema essa funcionalidade deve ficar?

SENAI

Exemplo — diagrama de componentes

Organização da consulta de status.

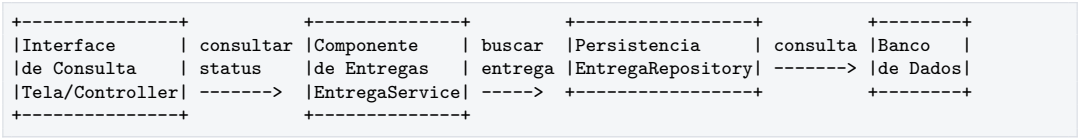


Diagrama de componentes mostra onde a funcionalidade está organizada dentro da arquitetura.

SENAI

Síntese dos diagramas de classes, sequência e componentes

Cada diagrama responde a uma pergunta diferente.

- **Diagrama de classes:** o que existe no sistema?
- **Diagrama de sequência:** o que acontece durante a execução?
- **Diagrama de componentes:** onde a funcionalidade está organizada?
- Os três diagramas podem representar a mesma funcionalidade observada de pontos de vista diferentes.

Não é fazer vários diagramas por fazer; cada diagrama deve ajudar a explicar uma parte da solução.

SENAI

Arquitetura e design trabalham juntos

Arquitetura não deve ficar separada do código.

- Arquitetura define grandes decisões, componentes e características importantes.
- Design detalha classes, interfaces, operações e colaboração entre objetos.
- Código materializa essas decisões.
- A equipe precisa manter ida e volta entre decisão e implementação.

Não basta desenhar arquitetura; a implementação precisa respeitá-la.

SENAI

Da interface para responsabilidades

Cada ação na interface costuma exigir responsabilidades no sistema.

- Botão “Consultar”: alguém precisa receber a solicitação.
- Campo “Código”: alguém precisa validar.
- Resultado “Status”: alguém precisa buscar e formatar.
- Mensagem de erro: alguém precisa tratar exceção ou ausência de dados.

Interface sem responsabilidade vira desenho; responsabilidade sem interface vira código difícil de usar.

SENAI

Exemplo de responsabilidades

Consulta de status da entrega.

Parte	Responsabilidade
Tela/Controller	Receber código, chamar serviço e mostrar resposta.
Service	Aplicar regra de consulta e tratamento de status.
Repository	Buscar dados da entrega.
Modelo Entrega	Representar dados e estado da entrega.

A tabela ajuda a transformar tela em estrutura de código.

SENAI

Atividades em sala - Mermaid no GitHub

Diagramas podem ficar dentro do README.

```
flowchart LR
  A[Cliente informa codigo] --> B[Sistema valida pedido]
  B --> C[Busca status da entrega]
  C --> D[Mostra status na interface]
```

- O diagrama fica junto da documentação.
- Mudanças aparecem no histórico de commits.
- É simples para fluxos, sequência e classes básicas.

Mermaid é útil quando a documentação também precisa ser versionada.

SENAI

Rastreabilidade

É o fio que liga tudo que foi feito.



- Mostra que o código não surgiu aleatoriamente.
- Ajuda o professor a avaliar coerência.
- Ajuda a equipe a encontrar lacunas.

Sem rastreabilidade, a entrega parece um conjunto de partes soltas.

SENAI

Tabela de rastreabilidade mínima

Use uma tabela simples no README ou em arquivo Markdown.

Req.	Interface	Modelo	Arquitetura	Código
RF-08-01	Tela consulta status	Fluxo/seq. consulta	Componente Entregas	consultarStatus

- Uma linha já pode ser suficiente para a fatia mínima.
- O importante é que cada célula seja verificável.

A tabela deve apontar onde a evidência está no projeto.

SENAI

Funcionalidade executável

A implementação pode ser simples, mas precisa rodar.

- Pode ser uma tela, endpoint, função, método, comando ou teste demonstrável.
- Se alterar código, compile antes de concluir.
- Se houver teste, execute o teste.
- Se não houver interface visual pronta, demonstre pelo terminal ou teste automatizado.

A evidência mínima é: código alterado + execução sem erro + commit.

SENAI

Testes e validação

Testar é confirmar que o requesito funciona.

- Teste o caminho principal: consulta válida.
- Teste um caminho alternativo: código inexistente.
- Registre como confirmou o funcionamento.
- Evite entregar código que não compila.

Validação simples é melhor que promessa de funcionamento.

SENAI

GitHub como evidência

A entrega precisa deixar rastro verificável.

- Issue: descreve problema e tarefa.
- Branch: isola a implementação da aula.
- Commits: mostram evolução do trabalho.
- Pull Request: organiza revisão e fechamento.
- README: centraliza explicação, diagramas e rastreabilidade.

Mensagem sugerida: aula08-fluxo-interface-modelo-arquitetura-codigo.

SENAI

Critérios de avaliação da atividade

O professor avaliará coerência, não quantidade de arquivos.

- Requisito escolhido é claro e pequeno.
- Fluxo/jornada representa o uso da funcionalidade.
- Wireframe/protótipo combina com o fluxo.
- UML/Mermaid explica o modelo relacionado.
- Componente arquitetural foi identificado.
- Código executa ou evolui funcionalidade existente.
- Rastreabilidade liga tudo de ponta a ponta.

A pergunta final é: dá para entender o caminho do requisito até o código?

SENAI

Erros comuns

Evite transformar a aula em desenho solto ou código sem explicação.

- Fazer tela sem requisito.
- Fazer requisito sem fluxo.
- Fazer diagrama que não aparece no código.
- Implementar em qualquer classe sem responsabilidade clara.
- Não compilar nem testar.
- Não registrar issue, branch e commits.

O erro principal é perder a coerência entre artefatos.

SENAI

Aplicação no projeto do grupo

Cada equipe deve usar nomes e exemplos do próprio domínio.

- Projeto de hospital: agendar consulta, buscar paciente, registrar atendimento.
- Projeto educacional: consultar turma, lançar atividade, visualizar nota.
- Projeto de farmácia: verificar estoque, registrar venda, consultar pedido.
- Projeto de estacionamento: liberar vaga, registrar entrada, calcular permanência.

Não copie NexusLog; use NexusLog apenas como exemplo de raciocínio.

SENAI

Roteiro prático para a equipe

Siga esta ordem durante a segunda parte da aula.

- 1. Escolher uma situação real do projeto.
- 2. Registrar problema e requisito.
- 3. Desenhar fluxo/jornada.
- 4. Criar wireframe simples.
- 5. Criar modelo UML/Mermaid relacionado.
- 6. Identificar componente arquitetural.
- 7. Implementar ou evoluir uma funcionalidade mínima.
- 8. Compilar, testar e registrar evidências no GitHub.

Use a ordem para não pular do problema direto para o código.

SENAI

Checklist de entrega da Aula 08

Antes de apresentar, confirme todos os itens.

- Um requisito selecionado.
- Fluxo ou jornada do usuário.
- Wireframe ou protótipo simples.
- Modelo UML/Mermaid relacionado.
- Componente arquitetural envolvido.
- Funcionalidade executável ou evolução de funcionalidade existente.
- Rastreabilidade requisito -> interface -> modelo -> arquitetura -> código.

A entrega deve estar dentro da pasta atividades ou docs, conforme organização do projeto.

Exemplo completo resumido

NexusLog - acompanhamento de entrega.

Problema	Cliente não sabe onde está a entrega.
Requisito	Consultar status pelo código do pedido.
Fluxo	Informar código -> validar -> buscar status -> exibir resultado.
Interface	Tela com campo de código, botão consultar e área de resultado.
Modelo	Sequência entre Tela, EntregaService e Repository.
Arquitetura	Componente de Entregas.
Código	Método consultarStatus e teste de consulta.

Esse é o padrão de raciocínio esperado nos projetos dos grupos.

SENAI

Ligação com a prova N1

A aula ajuda a revisar vários assuntos já estudados.

- Requisitos funcionais e não funcionais.
- Coesão, acoplamento e responsabilidades.
- Modelagem UML/Mermaid.
- Componentes, conectores e arquitetura.
- Documentação, decisão e rastreabilidade.
- Implementação como evidência de coerência do design.

A prática da Aula 08 prepara a revisão integrada da Aula 09.

SENAI

Fechamento

Design de software é ligar decisões, modelos e código para resolver problemas reais.

- Comece pelo **problema do usuário**.
- Escolha um **requisito pequeno**.
- **Modele** apenas **o necessário**.
- **Identifique responsabilidades** e **componente** arquitetural.
- **Implemente uma fatia mínima** executável.
- **Rastreie tudo** para a própria equipe entenderem ou outros que lerem o projeto.

Próxima aula: Aula 09 - revisão integrada para a prova N1.

SENAI

Próximos passos

- Na Aula 09, revisaremos integrada dos conceitos para prova.
- Tragam dúvidas e lacunas encontradas sobre conceitos.
- A prova N1 ocorrerá na Aula 10, conforme planejamento da disciplina.

Hoje: fechar a fatia vertical. Próxima aula: revisar para a N1.